Implement Greedy search algorithm for any of the following application: I. Selection Sort

Explanation for Oral:

- Selection Sort divides the array into sorted and unsorted parts.
- In every pass, the smallest element is selected from the unsorted part.
- That element is placed in its correct position using swapping.
- Since it greedily chooses the minimum element at each step, it is called a greedy algorithm.
- Time Complexity:
    - Best Case: $O(n^2)$
    - Average Case: $O(n^2)$
    - Worst Case: $O(n^2)$
- Space Complexity: O(1) because sorting is done in-place.

Theory:
Selection Sort is a greedy algorithm because at every step it selects the minimum element from the unsorted part of the array and places it at the correct position. It makes the locally optimal choice in each iteration.

Algorithm:

1. Start from the first element.
2. Find the smallest element in the remaining unsorted array.
3. Swap it with the current element.
4. Repeat until the array is sorted.

```
def selection(arr):
    n = len(arr)

    for i in range(n - 1):
        mini = i

        # Find minimum element
        for j in range(i + 1, n):
            if arr[j] < arr[mini]:
                mini = j

        # Swap elements
        arr[i], arr[mini] = arr[mini], arr[i]



# ---- Main Program ----
n = int(input("Enter number of elements: "))
arr = []

for i in range(n):
    num = int(input(f"Enter element {i+1}: "))
    arr.append(num)

print("Original Array:", arr)

selection(arr)

print("Sorted Array is:", arr)
```